

Chapter 6: Recognising C Code Constructs in Assembly

1. Introduction

This manual explains how common C programming constructs appear after compilation into assembly language. The practical exercises demonstrate how programs can be analysed using IDA Pro after compiling C source code with GCC.

The chapter focuses on:

- Global and local variables
- Arithmetic operations
- If statements
- Nested if statements
- Loops
- Function call conventions
- Switch statements
- Arrays
- Graph analysis in IDA Pro

The exercises are designed to strengthen reverse engineering and malware analysis skills by helping students recognise common assembly patterns generated by C programs.

2. Setting Up the Lab Environment Using Dev-C++

Objective

The purpose of this practical exercise is to understand how global and local variables are represented in assembly language after a C program is compiled. The exercise also introduces the process of compiling C programs and analysing executables in IDA Pro.

Before analysing assembly code in IDA Pro, a small lab environment was prepared to allow C programs to be written, compiled, executed, and reverse engineered.

The lab environment consisted of:

- Windows 10 Virtual Machine
- Dev-C++
- GCC Compiler (included with Dev-C++)
- IDA Free / IDA Pro

Installing Dev-C++

Dev-C++ was used as the code editor and compiler environment for writing, compiling, and running C programs.

Dev-C++ includes the GCC compiler, which means a separate compiler installation is usually not required.

Dev-C++ can be downloaded from:

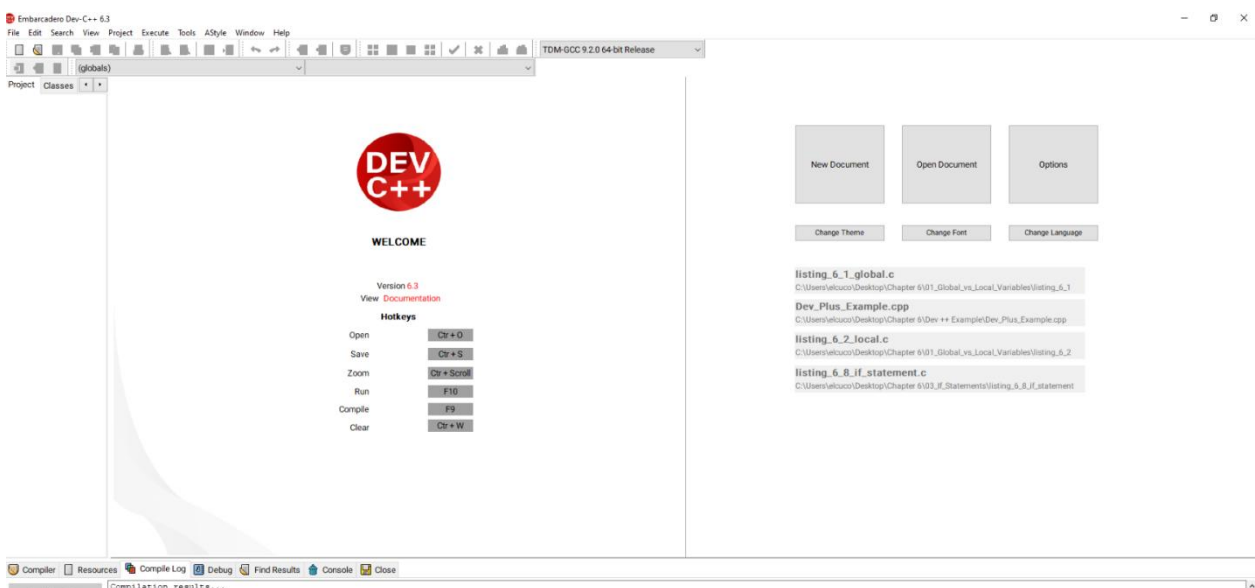
[Embarcadero Dev-C++ Download](#)

After downloading:

1. Run the installer
2. Follow the installation steps
3. Select the default GCC compiler package during installation
4. Complete the setup

Once installed, Dev-C++ provides:

- C/C++ code editing
- Syntax highlighting
- Built-in GCC compiler
- Program compilation and execution
- Debugging support



Verifying the Compiler Installation

To verify that the GCC compiler is working correctly:

1. Open Dev-C++
2. Create a new source file
3. Type the following program:

```
#include <stdio.h>

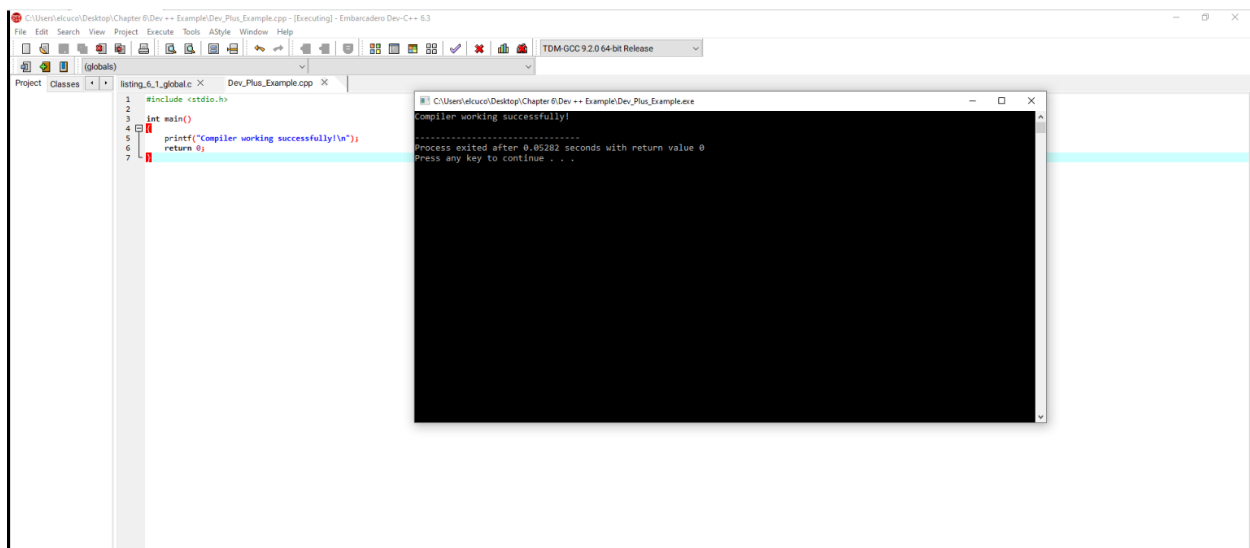
int main()
{
    printf("Compiler working successfully!\n");
    return 0;
}
```

4. Save the file
5. Press:

F11 or select: Execute → Compile & Run

If the program runs successfully, the compiler is correctly installed.

Dev-C++ automatically generates the executable (.exe) file in the project folder.



Installing IDA Free / IDA Pro

IDA Free / IDA Pro was used for static analysis and disassembly of executable files.

IDA converts machine code into assembly language instructions, allowing analysis of:

- Variables
- Memory usage
- Arithmetic operations
- Function calls
- Stack operations

IDA Free can be downloaded from:

<https://hex-rays.com/>

After installation, the compiled executable files were opened in IDA for reverse engineering and assembly analysis.

3. Global vs Local Variables

a) Create the Source Code File

- Create the following folder structure on the **Desktop**:

```
Chapter 6
├── 01_Global_vs_Local_Variables
│   └── listing_6_1_global
```

- Launch Visual Studio Code or a text editor of your choice.

Inside the folder, create a new file named:

- `Listing_6_1_global.c`

Type the following C program exactly as shown below:

```
#include <stdio.h>

int x = 1;
int y = 2;

int main()
{
    x = x + y;
    printf("total = %d\n", x);
}
```

```
    return 0;  
}
```

b) What this means

`x` and `y` are **global variables** because they are declared outside `main()`.

They start as:

`x = 1`

`y = 2`

Then this line runs:

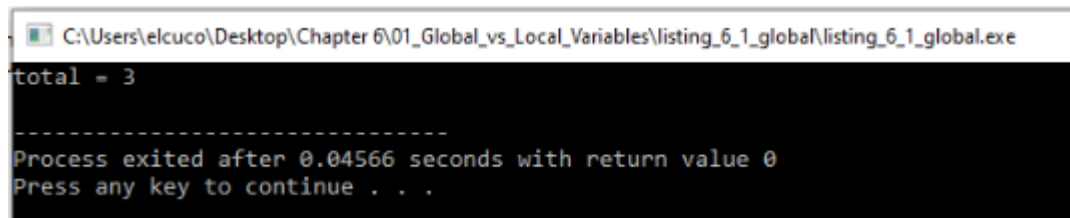
`x = x + y;`

So: `x = 1 + 2`

`x = 3`

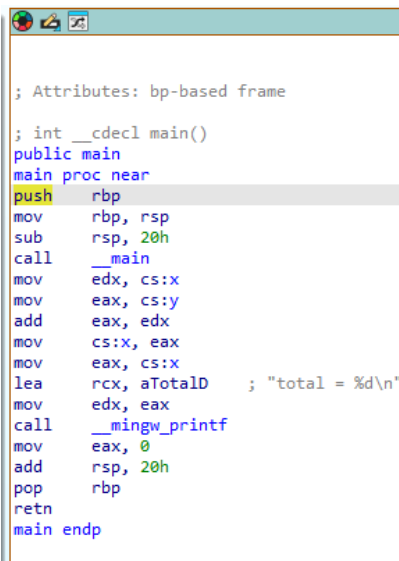
Then this prints: `printf("total = %d\n", x);`

Output: total = 3



```
C:\Users\elcuco\Desktop\Chapter 6\01_Global_vs_Local_Variables\listing_6_1_global\listing_6_1_global.exe
total = 3
-----
Process exited after 0.04566 seconds with return value 0
Press any key to continue . . .
```

c) Your IDA assembly



```
; Attributes: bp-based frame
; int __cdecl main()
public main
main proc near
push    rbp
mov     rbp, rsp
sub     rsp, 20h
call    __main
mov     edx, cs:x
mov     eax, cs:y
add     eax, edx
mov     cs:x, eax
mov     eax, cs:x
lea     rcx, aTotalD ; "total = %d\n"
mov     edx, eax
call    __mingw_printf
mov     eax, 0
add     rsp, 20h
pop     rbp
retn
main endp
```

Focus only on these important lines:

```
mov     edx, cs:x
mov     eax, cs:y
add     eax, edx
mov     cs:x, eax
mov     eax, cs:x
lea     rcx, aTotalD
mov     edx, eax
call    __mingw_printf
```

d) Explanation

In this example, **x** and **y** are global variables because they are declared outside the main function. In the assembly code, IDA references them directly as **cs:x** and **cs:y**.

This shows that the variables are stored in a fixed memory location, not on the stack.

The instruction **mov edx, cs:x** loads the value of **x** into **edx**. Since **x** is **1**, **edx** becomes **1**. The instruction **mov eax, cs:y** loads the value of **y** into **eax**. Since **y** is **2**, **eax** becomes **2**.

The instruction **add eax, edx** adds the two values together. After this instruction, **eax** contains **3**.

The instruction **mov cs:x, eax** stores the result back into the global variable **x**. This means **x** has now changed from **1** to **3** in memory.

The next instructions prepare the printf function. The string “total = %d\n” is loaded, and the updated value of **x** is passed to printf. The call **__mingw_printf** instruction then prints the result on the screen.

e) Your key point

The important observation is that global variables are referenced directly by memory names such as **cs:x** and **cs:y**. This is different from local variables, which are referenced using stack offsets such as **[rbp-4]**.

4. Local Variables in Assembly Language

a) Create the Source Code File

- Create the following folder structure on the Desktop:

Chapter 6

└─ 01_Global_vs_Local_Variables

└─ listing_6_2_local

- Launch Visual Studio Code or a text editor of your choice.

Inside the folder, create a new file named:

Listing_6_2_local.c

Type the following C program exactly as shown below:

```
#include <stdio.h>

int main()
{
    int x = 1;

    int y = 2;

    x = x + y;

    printf("total = %d\n", x);

    return 0;
}
```


b) What this means

`x` and `y` are local variables because they are declared inside `main()`.

They start as: `x = 1` `y = 2`

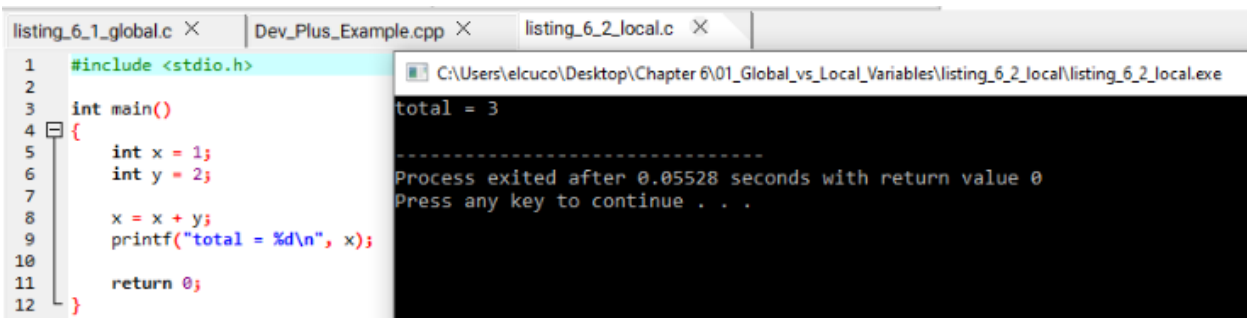
Then this line runs: `x = x + y;`

So: `x = 1 + 2`

`x = 3`

Then this prints: `printf("total = %d\n", x);`

Output: `total = 3`



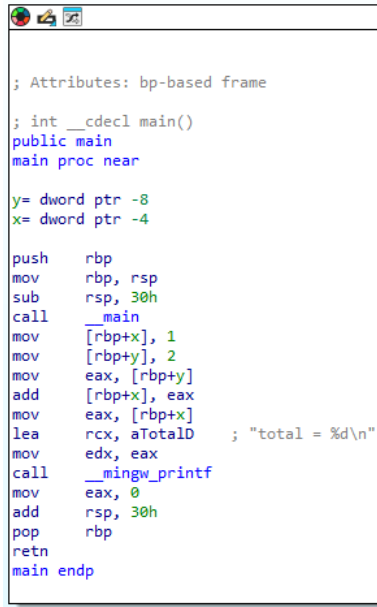
The screenshot shows a code editor with three tabs: `listing_6_1_global.c`, `Dev_Plus_Example.cpp`, and `listing_6_2_local.c`. The `listing_6_2_local.c` tab is active, displaying the following C code:

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int x = 1;
6     int y = 2;
7
8     x = x + y;
9     printf("total = %d\n", x);
10
11     return 0;
12 }
```

To the right of the code editor is a terminal window showing the output of the program. The output is:

```
total = 3
-----
Process exited after 0.05528 seconds with return value 0
Press any key to continue . . .
```

c) Your IDA Assembly



```
; Attributes: bp-based frame
; int __cdecl main()
public main
main proc near

y= dword ptr -8
x= dword ptr -4

push    rbp
mov     rbp, rsp
sub     rsp, 30h
call    __main
mov     [rbp+x], 1
mov     [rbp+y], 2
mov     eax, [rbp+y]
add     [rbp+x], eax
mov     eax, [rbp+x]
lea     rcx, aTotalD    ; "total = %d\n"
mov     edx, eax
call    __mingw_printf
mov     eax, 0
add     rsp, 30h
pop     rbp
retn
main endp
```

Focus only on these important lines:

```
mov     [rbp+x], 1
mov     [rbp+y], 2
mov     eax, [rbp+y]
add     [rbp+x], eax
mov     eax, [rbp+x]
lea     rcx, aTotalD
mov     edx, eax
call    __mingw_printf
```

IDA also identifies the local variables as:

```
y = dword ptr -8
x = dword ptr -4
```

d) 4. Explanation

In this example, `x` and `y` are local variables because they are declared inside the `main()` function. Unlike global variables, they are not referenced directly by memory names such as `cs:x` or `cs:y`.

Instead, the variables are stored on the stack and referenced using `rbp`-relative offsets.

The instruction: `mov [rbp+x], 1`

stores the value `1` into the local variable `x`.

The instruction:

`mov [rbp+y], 2` stores the value `2` into the local variable `y`.

The instruction: `mov eax, [rbp+y]` loads the value of `y` into `eax`. Since `y` is `2`, `eax` becomes `2`.

The instruction: `add [rbp+x], eax` adds the value stored in `eax` to the local variable `x`.

After this instruction: `x = 3`

The instruction: `mov eax, [rbp+x]`

loads the updated value of `x` into `eax` so it can be passed to `printf`.

The next instructions prepare the `printf` function. The string `"total = %d\n"` is loaded, and the updated value of `x` is passed to `printf`.

The instruction:

`call __mingw_printf` then prints the result on the screen.

e) Your key point

The important observation is that local variables are referenced using stack offsets such as:

`[rbp+x]`

`[rbp+y]`

rather than direct memory references such as:

`cs:x`

This indicates that local variables are stored temporarily on the stack and exist only while the function is executing.

5. Disassembling Arithmetic Operations

a) Create the Source Code File

Create the following folder structure on the Desktop:

```
Chapter 6
├── 02_Arithmetic_Operations
│   └── listing_6_6_arithmetic
```

Launch Visual Studio Code or a text editor of your choice.

Inside the folder, create a new file named:

```
listing_6_6_arithmetic.c
```

Type the following C program exactly as shown below:

```
#include <stdio.h>

int main()
{
    int a = 0;
    int b = 1;

    a = a + 11;
    a = a - b;
    a--;
    b++;

    b = a % 3;

    return 0;
}
```

b) What this means

The program creates two local variables:

```
a = 0
b = 1
```

The program then performs several arithmetic operations.

Addition

```
a = a + 11;      This increases a by 11.
```

Result: $a = 11$

Subtraction

```
a = a - b;      This subtracts b from a.
```

Result: $a = 10$

because: $11 - 1 = 10$

Decrement

```
a--;   This decreases a by 1.      Result:  $a = 9$ 
```

Increment

```
b++;      This increases b by 1.      Result:  $b = 2$ 
```

Modulo Operation

```
b = a % 3;   The % operator performs modulus division.
```

It stores the remainder after division. Since: $9 \% 3 = 0$

Result: $b = 0$

c) 3. Your IDA Assembly

Focus only on these important lines:

```
mov     [rbp+a], 0
mov     [rbp+b], 1

mov     eax, [rbp+a]
add     eax, 0Bh
mov     [rbp+a], eax

mov     ecx, [rbp+a]
sub     ecx, [rbp+b]
mov     [rbp+a], ecx

mov     edx, [rbp+a]
sub     edx, 1
mov     [rbp+a], edx

mov     eax, [rbp+b]
add     eax, 1
mov     [rbp+b], eax
```

d) Explanation

In this example, the variables **a** and **b** are local variables stored on the stack.

The instruction: `mov [rbp+a], 0` initialises the local variable **a** with the value **0**.

The instruction: `mov [rbp+b], 1` initialises the local variable **b** with the value **1**.

Addition Operation

The instructions:

```
mov eax, [rbp+a]
add eax, 0Bh
mov [rbp+a], eax
```

perform the operation: $a = a + 11;$

The value of **a** is loaded into **eax**, **11** is added, and the result is stored back into **a**.

0Bh is hexadecimal for decimal **11**.

Subtraction Operation

The instructions:

```
mov ecx, [rbp+a]
sub ecx, [rbp+b]
mov [rbp+a], ecx
```

perform the operation: $a = a - b;$

The value of **b** is subtracted from **a**, and the updated result is stored back into **a**.

Decrement Operation

The instructions:

```
mov edx, [rbp+a]
sub edx, 1
mov [rbp+a], edx
```

perform the operation: $a--;$

This decreases **a** by 1.

Increment Operation

The instructions:

```
mov eax, [rbp+b]
add eax, 1
mov [rbp+b], eax
```

perform the operation: $b++;$

This increases **b** by 1.

e) 5. Your key point

The important observation is that arithmetic operations in C are translated into assembly instructions such as:

add AND sub

The compiler loads values into CPU registers, performs the arithmetic operation, and then stores the updated result back into memory.

This is one of the most common patterns observed during reverse engineering and malware analysis.

6. Recognising If Statements in Assembly Language

a) Create the Source Code File

Create the following folder structure on the Desktop:

```
Chapter 6
└─ 03_If_Statements
    └─ listing_6_8_if_statement
```

Inside the folder, create a new file named the following:

```
listing_6_8_if_statement.c
```

Type the following C program:

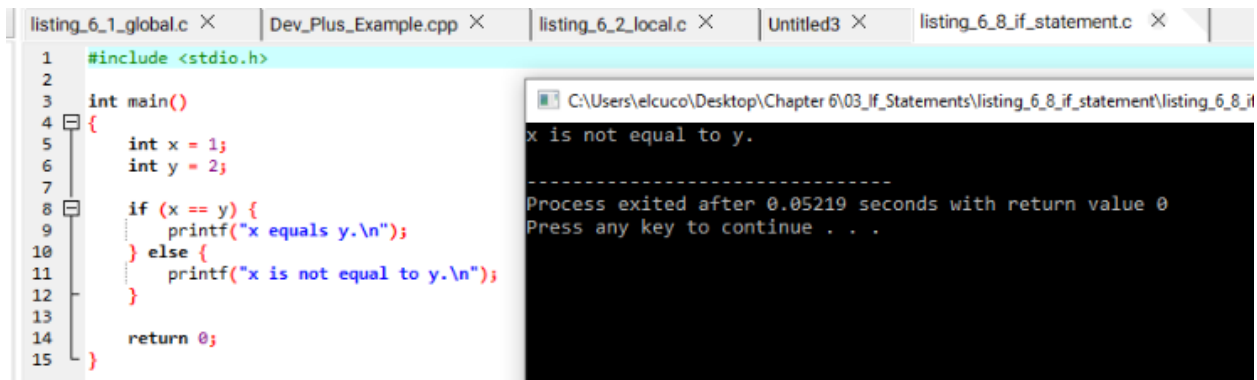
```
#include <stdio.h>

int main()
{
    int x = 1;
    int y = 2;

    if (x == y) {
        printf("x equals y.\n");
    } else {
        printf("x is not equal to y.\n");
    }

    return 0;
}
```

Compile and Run it



The screenshot shows a code editor with five tabs: listing_6_1_global.c, Dev_Plus_Example.cpp, listing_6_2_local.c, Untitled3, and listing_6_8_if_statement.c. The active tab is listing_6_8_if_statement.c, which contains the following C code:

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int x = 1;
6      int y = 2;
7
8      if (x == y) {
9          printf("x equals y.\n");
10     } else {
11         printf("x is not equal to y.\n");
12     }
13
14     return 0;
15 }
```

To the right of the code editor is a terminal window titled "C:\Users\elcuco\Desktop\Chapter 6\03_If_Statements\listing_6_8_if_statement\listing_6_8_if_statement.exe". The terminal output shows:

```
x is not equal to y.
-----
Process exited after 0.05219 seconds with return value 0
Press any key to continue . . .
```

b) What this means

The program creates two local variables: $x = 1$ and $y = 2$

Then it checks this condition: `if (x == y)`

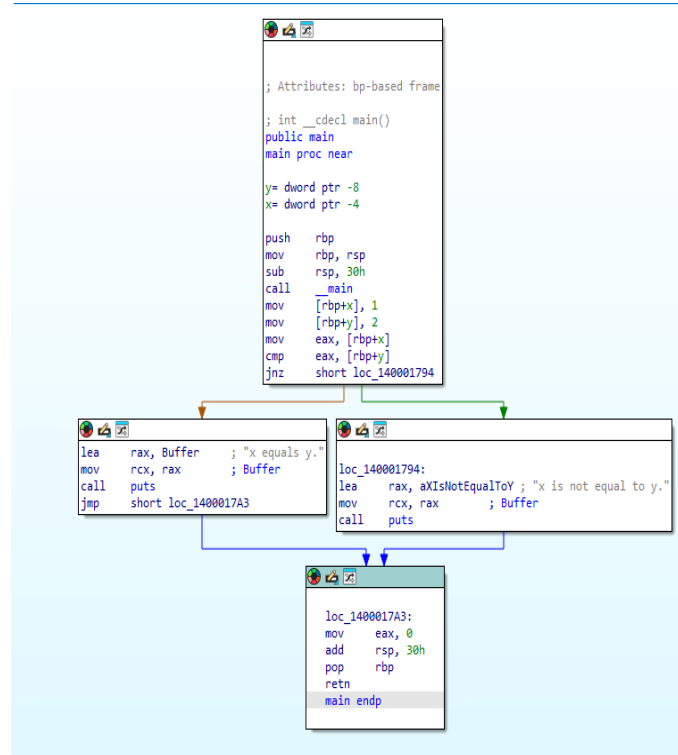
This means: Is x equal to y ?

Since: 1 is not equal to 2

the condition is false.

Therefore, the program executes the **else** part: `printf("x is not equal to y.\n");`

c) 3. Your IDA Assembly



```

.text:0000000140001768      call    __main
.text:000000014000176D      mov     [rbp+x], 1
.text:0000000140001774      mov     [rbp+y], 2
.text:0000000140001778      mov     eax, [rbp+x]
.text:000000014000177E      cmp     eax, [rbp+y]
.text:0000000140001781      jnz     short loc_140001794
.text:0000000140001783      lea     rax, Buffer ; "x equals y."
.text:000000014000178A      mov     rcx, rax ; Buffer
.text:000000014000178D      call    puts
.text:0000000140001792      jmp     short loc_1400017A3
.text:0000000140001794 ; -----
.text:0000000140001794      .text:0000000140001794 loc_140001794: ; CODE XREF: main+21↑j
.text:0000000140001794      lea     rax, aXIsNotEqualToY ; "x is not equal to y."
.text:0000000140001798      mov     rcx, rax ; Buffer
.text:000000014000179E      call    puts
.text:00000001400017A3      .text:00000001400017A3 loc_1400017A3: ; CODE XREF: main+32↑j
.text:00000001400017A3      mov     eax, 0
.text:00000001400017A8      add     rsp, 30h
.text:00000001400017AC      pop     rbp
.text:00000001400017AD      retn
.text:00000001400017AD      main      endp
.text:00000001400017AD

```

d) Explanation

In this example, `x` and `y` are local variables because they are declared inside the `main()` function. IDA references them using stack offsets such as `[rbp+x]` and `[rbp+y]`.

`mov eax, [rbp+x]` loads the value of `x` into the register `eax`.

`cmp eax, [rbp+y]` compares the value of `x` with the value of `y`.

`jnz short loc_else` means **jump if not zero**. In this case, it means: If `x` is not equal to `y`, jump to the `else` block.

Since `x = 1` and `y = 2`, the values are not equal. Therefore, the jump is taken, and the program moves to the `else` section.

The `else` section then prints: `x is not equal to y`.

e) Your key point

The important observation is that an `if` statement is usually recognised in assembly by:

`cmp` conditional jump

For example:

```
cmp     eax, [rbp+y]
jnz     short loc_else
```

This shows that the program is making a decision based on a condition.

In reverse engineering, conditional jumps such as `jnz`, `jz`, `je`, `jne`, `jl`, and `jle` are strong indicators of decision-making logic in a program.

f) Analyzing Functions Graphically with IDA Pro

IDA Pro provides a graphing feature that helps visualise program execution flow. This feature is useful for recognising program constructs such as:

- if statements
- loops
- function calls
- nested conditions

The graph view allows reverse engineers to follow different execution paths visually instead of reading assembly instructions line by line.

Switching to Graph View in IDA

After opening the function in IDA:

1. Place the cursor inside the function
2. Press:

Spacebar

IDA will switch from text view to graph view.

Pressing the spacebar again switches back to normal assembly view.

Understanding the Graph View

In the graph view, each block represents a section of assembly instructions.

Arrows between the blocks show the possible execution paths of the program.

In the if statement example, the program has two execution paths:

Path	Meaning
Left path	<code>x equals y.</code>
Right path	<code>x is not equal to y.</code>

The decision is controlled by the conditional jump instruction:

```
jnz short loc_140001794
```

Understanding the Conditional Jump

The instruction: `cmp eax, [rbp+y]` compares the values of `x` and `y`.

The next instruction: `jnz short loc_140001794` means: Jump if values are NOT equal

Execution Paths

Path 1 — Condition is False

If `x` equals `y`:

- the jump is NOT taken
- execution continues downward

This path prints: `x equals y`.

Path 2 — Condition is True

If `x` does NOT equal `y`:

- the jump IS taken
- execution moves to the right-side block

This path prints: `x is not equal to y`.

Why Graph View Is Important

Graph view simplifies reverse engineering because it:

- visually separates execution paths
- makes conditions easier to recognise
- helps identify loops and nested logic
- speeds up analysis of complex programs

Instead of mentally tracking jumps, the analyst can visually follow the control flow.

Your key point

The important observation is that graph view visually represents conditional execution paths created by instructions such as:

```
cmp, jnz, jz, je, jne
```

This makes it easier to recognise decision-making logic during reverse engineering and malware analysis.

7. Recognising Nested If Statements in Assembly Language

a) Create the Source Code File

Create the following folder structure on the Desktop:

```
Chapter 6
├── 04_Nested_If_Statements
│   └── listing_6_10_nested_if
```

Inside the folder, create a new file named:

```
listing_6_10_nested_if.c
```

Type the following C program exactly as shown below:

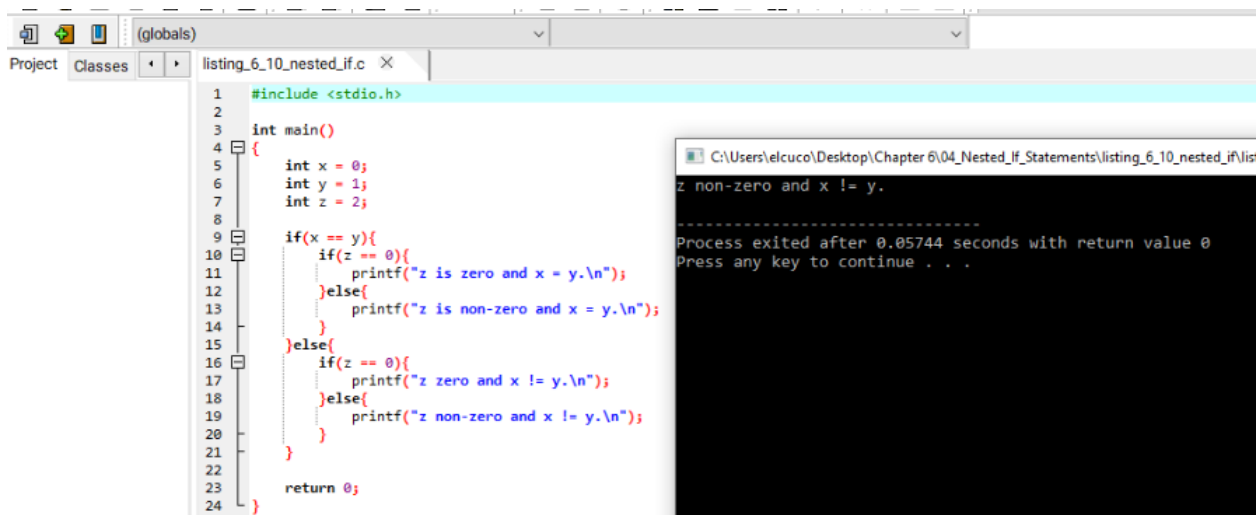
```
#include <stdio.h>

int main()
{
    int x = 0;
    int y = 1;
    int z = 2;

    if(x == y){
        if(z == 0){
            printf("z is zero and x = y.\n");
        }else{
            printf("z is non-zero and x = y.\n");
        }
    }else{
        if(z == 0){
            printf("z zero and x != y.\n");
        }else{
            printf("z non-zero and x != y.\n");
        }
    }

    return 0;
}
```

Compile and Run it



```
1 #include <stdio.h>
2
3 int main()
4 {
5     int x = 0;
6     int y = 1;
7     int z = 2;
8
9     if(x == y){
10         if(z == 0){
11             printf("z is zero and x = y.\n");
12         }else{
13             printf("z is non-zero and x = y.\n");
14         }
15     }else{
16         if(z == 0){
17             printf("z zero and x != y.\n");
18         }else{
19             printf("z non-zero and x != y.\n");
20         }
21     }
22
23     return 0;
24 }
```

```
z non-zero and x != y.
-----
Process exited after 0.05744 seconds with return value 0
Press any key to continue . . .
```

b) What this means

The program creates three local variables:

```
x = 0
y = 1
z = 2
```

The program first checks:

```
if(x == y)           Since:           0 != 1
```

the condition is false.

Execution moves to the **else** block.

Inside the **else** block, the program checks:

```
if(z == 0)           Since:           z = 2 the condition is also false.
```

Therefore, the program executes:

```
printf("z non-zero and x != y.\n");
```


c) What to Look for in IDA

When you open the executable in IDA, you will notice:

- multiple `cmp` instructions
- multiple conditional jumps
- several execution paths in graph view

Important instructions usually include:

```
cmp
jz
jnz
je
jne
```

Nested if statements create:

- multiple decision points
- multiple branches
- more complex control flow graphs

d) Explanation

In this example, the program contains an `if` statement inside another `if` statement. This is called a nested if statement.

The first comparison checks whether: `x == y`

IDA translates this into:

- a `cmp` instruction
- followed by a conditional jump

If the condition is false, execution jumps to the outer `else` block.

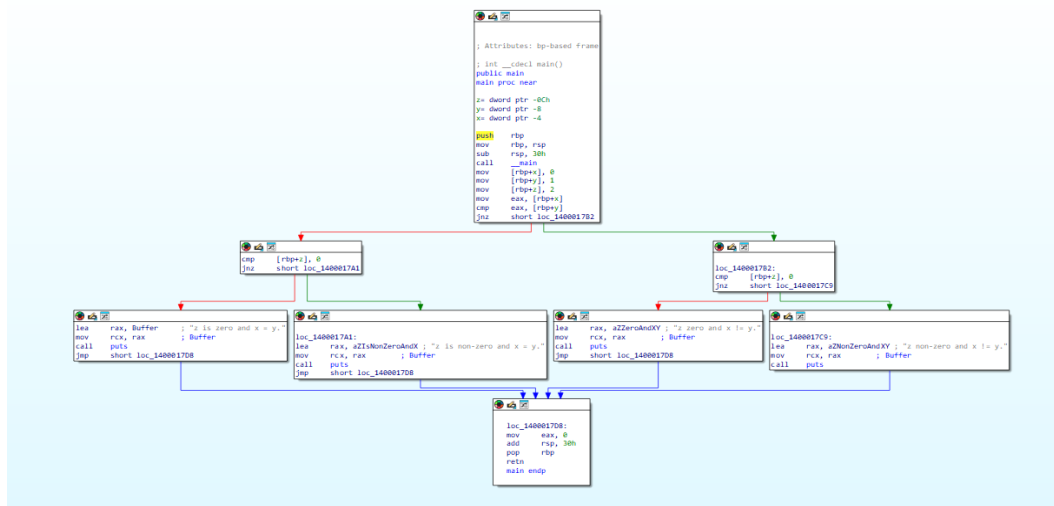
Inside each block, another comparison checks whether:

```
z == 0
```

This creates another decision point and another conditional jump.

Because nested if statements contain several conditions, the assembly becomes more complex and produces multiple execution paths in IDA graph view.

e) 5. Graph View Observation



When viewed in IDA graph mode:

- each condition creates a branch
- each branch leads to another decision or output
- the graph becomes larger than a simple if statement

You will notice several connected blocks representing:

- outer if
- inner if
- else sections
- final function exit

This visual structure helps identify nested decision-making logic during reverse engineering.

f) Your key point

The important observation is that nested if statements create multiple comparison and jump instructions in assembly.

A reverse engineer can recognise nested logic by identifying:

- repeated `cmp` instructions
- multiple conditional jumps
- several connected branches in graph view

Nested conditions significantly increase the complexity of a program's control flow.

8. Recognising Loops in Assembly Language

a) Finding For Loops

Loops are very common in software and malware analysis. A loop allows a program to repeat instructions multiple times until a condition becomes false.

A **for** loop normally contains four components:

- initialization
- comparison
- execution instructions
- increment or decrement

b) Create the Source Code File

Create the following folder structure on the Desktop:

```
Chapter 6
├── 05_Loops
│   └── listing_6_12_for_loop
```

Launch Visual Studio Code or a text editor of your choice.

Inside the folder, create a new file named:

```
listing_6_12_for_loop.c
```

Type the following C program exactly as shown below:

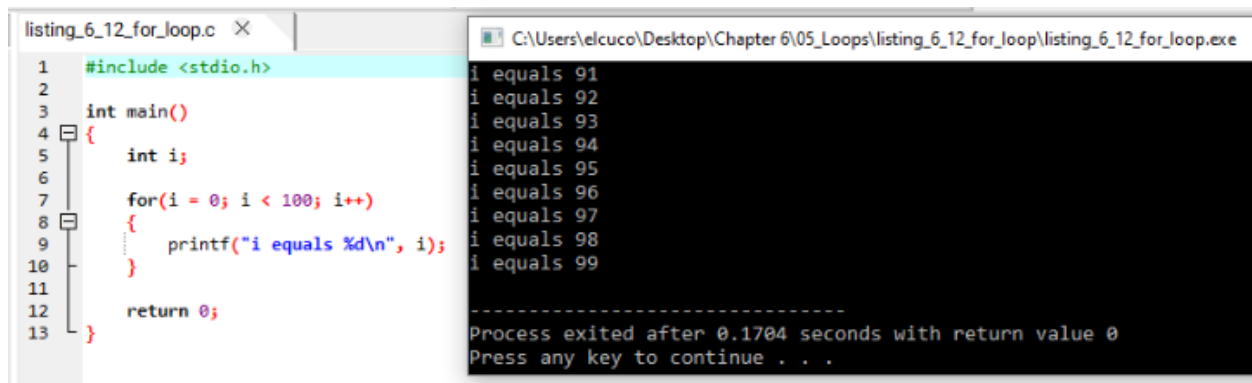
```
#include <stdio.h>

int main()
{
    int i;

    for(i = 0; i < 100; i++)
    {
        printf("i equals %d\n", i);
    }

    return 0;
}
```

Compile and Run it



```
listing_6_12_for_loop.c
1  #include <stdio.h>
2
3  int main()
4  {
5      int i;
6
7      for(i = 0; i < 100; i++)
8      {
9          printf("i equals %d\n", i);
10     }
11
12     return 0;
13 }
```

```
C:\Users\elcucu\Desktop\Chapter 6\05_Loops\listing_6_12_for_loop\listing_6_12_for_loop.exe
i equals 91
i equals 92
i equals 93
i equals 94
i equals 95
i equals 96
i equals 97
i equals 98
i equals 99

-----
Process exited after 0.1704 seconds with return value 0
Press any key to continue . . .
```

c) What this means

The program creates a local variable: `i`

The loop starts by setting: `i = 0`

Then the condition: `i < 100` is checked.

If the condition is true:

- the `printf` statement executes
- `i` increases by 1
- the condition is checked again

This process repeats until `i` becomes `100`.

At that point:

`100 < 100` is false, so the loop stops.

3. What to Look for in IDA

In assembly language, loops are recognised by:

- comparisons
- conditional jumps
- backward jumps

Important instructions usually include:

```
cmp
jl
jg
```

```
jmp
add
inc
```

A loop normally:

1. initializes a variable
2. compares it
3. executes code
4. jumps back to repeat

4. Explanation

The instruction that initializes the loop variable usually looks similar to:

```
mov [rbp+i], 0
```

This sets `i` to `0`. The comparison section usually contains:

```
cmp [rbp+i], 64h
```

`64h` is hexadecimal for decimal `100`.

This instruction checks whether `i` is less than `100`.

The loop body contains the `printf` call:

```
call __mingw_printf
```

which prints the current value of `i`.

The increment operation usually appears as:

```
add [rbp+i], 1
```

or:

```
inc [rbp+i]
```

This increases `i` by 1 after every iteration.

The important part of a loop is the backward jump instruction. After incrementing `i`, the program jumps back to the comparison section to repeat the process.

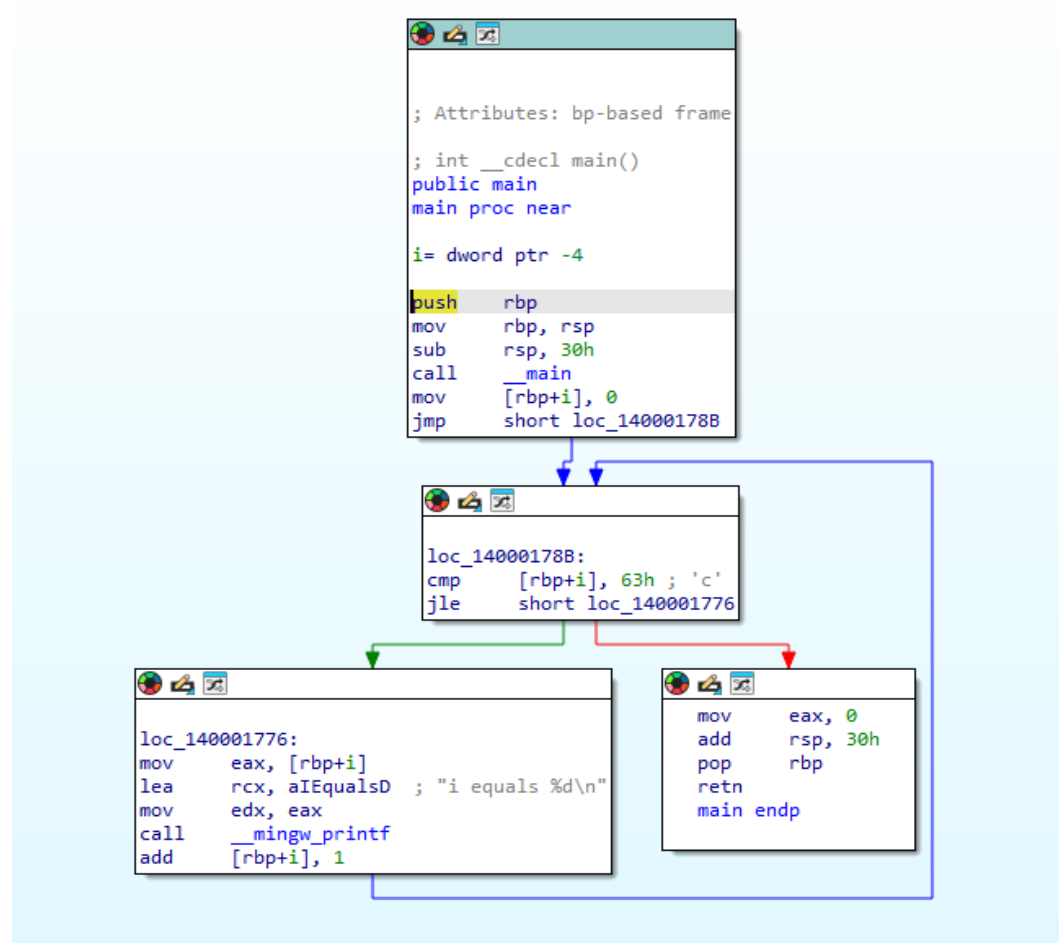
5. Graph View Observation

In IDA graph mode:

- loops appear as circular paths
- one branch returns to an earlier block
- this creates repetition in execution flow

Unlike an if statement, a loop repeatedly revisits the same block of code until the condition becomes false.

This looping structure is easy to recognise visually in graph view.



6. Your key point

The important observation is that loops are recognised in assembly by:

- comparisons
- conditional jumps
- backward jumps

A reverse engineer can identify loops by looking for execution paths that return to earlier instructions. This repeated control flow is one of the clearest indicators of looping behaviour in assembly language.

9. Finding While Loops in Assembly Language

1. Create the Source Code File

Create the following folder structure on the Desktop:

```
Chapter 6
├── 06_While_Loops
│   └── listing_6_14_while_loop
```

Inside the folder, create a new file named:

listing_6_14_while_loop.c

Type the following C program exactly as shown below:

```
#include <stdio.h>

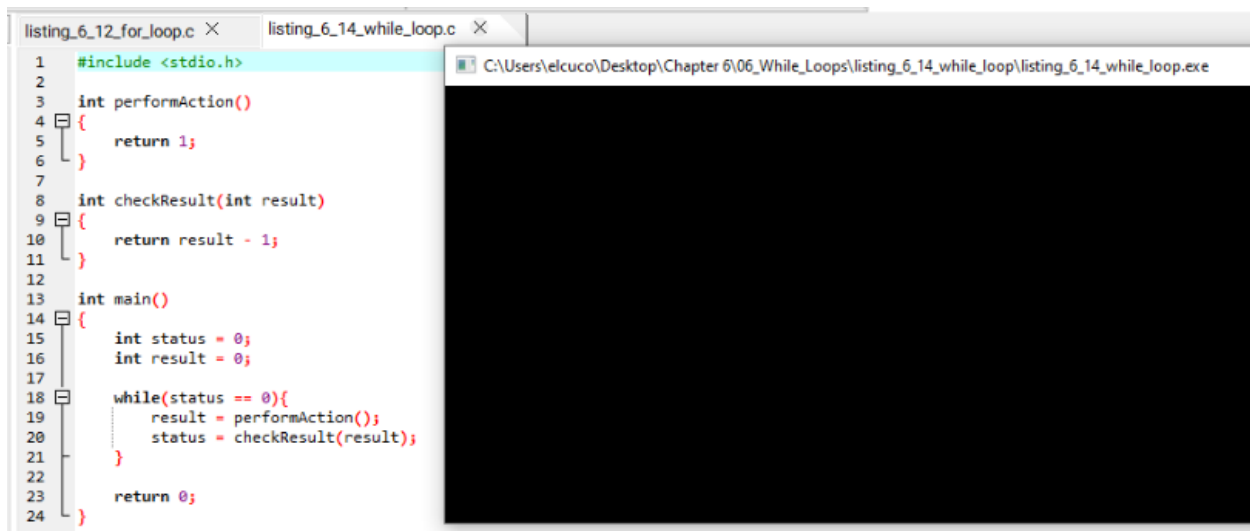
int performAction()
{
    return 1;
}

int checkResult(int result)
{
    return result - 1;
}

int main()
```

```
{  
  
    int status = 0;  
    int result = 0;  
  
    while(status == 0){  
        result = performAction();  
        status = checkResult(result);  
    }  
}
```

Compile and Run it



The program finishes silently because nothing is printed.

2. What this means

The program creates two local variables:

```
status = 0
result = 0
```

The loop condition is: `while (status == 0)`

This means:

- continue looping while `status` equals 0

Inside the loop:

1. `performAction()` executes
2. its return value is stored in `result`
3. `checkResult(result)` executes
4. the returned value is stored in `status`

The loop continues until:

```
status != 0
```

At that point, the condition becomes false and the loop stops.

3. What to Look for in IDA

In assembly language, a `while` loop usually contains:

- a comparison
- a conditional jump
- an unconditional backward jump

Important instructions usually include:

```
cmp
jz
jnz
jmp
call
```

Unlike a `for` loop, a `while` loop normally does not contain a dedicated increment section.

4. Explanation

The instruction:

```
mov [rbp+status], 0
```

 initializes the variable `status` to `0`.

The loop condition is translated into a comparison instruction similar to:

```
cmp [rbp+status], 0
```

This checks whether `status` equals `0`.

The conditional jump instruction determines whether execution continues inside the loop or exits the loop.

Inside the loop, function calls appear similar to:

```
call performAction  
call checkResult
```

The result returned from `performAction()` is stored in `result`.

The value returned from `checkResult(result)` is then stored back into `status`.

At the end of the loop body, the program performs an unconditional jump back to the comparison section to repeat the process.

This repeated backward jump is one of the clearest indicators of looping behaviour in assembly language.

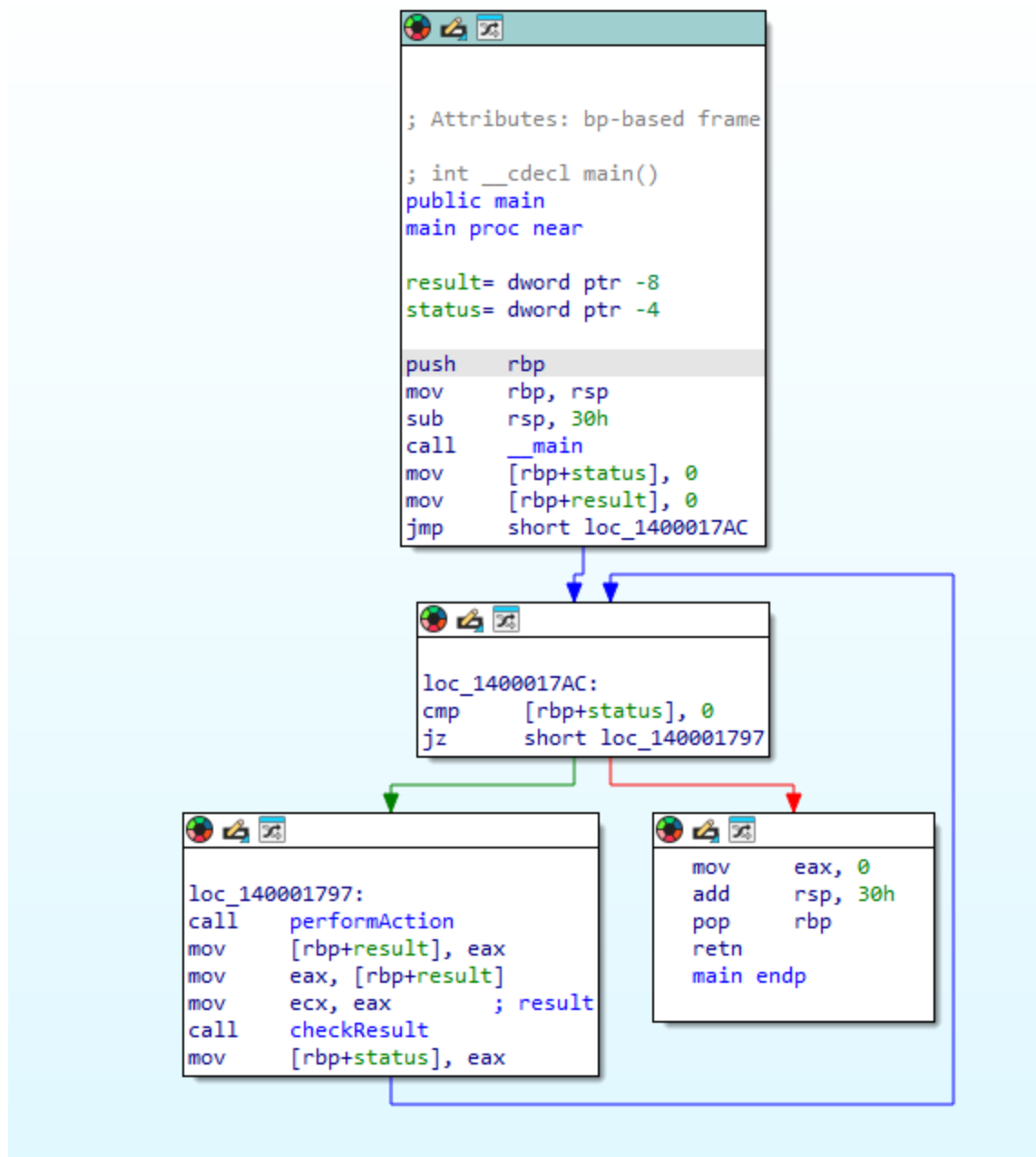
5. Graph View Observation

In IDA graph mode:

- the comparison block appears near the top
- one branch exits the loop
- another branch continues executing the loop body
- execution then jumps back to the comparison block

This creates a visible cycle in the control flow graph.

Unlike a simple if statement, the execution path repeatedly returns to the same comparison point.



6. Your key point

The important observation is that while loops are recognised in assembly by:

- a condition check
- conditional jumps
- unconditional backward jumps

A reverse engineer can identify while loops by following execution paths that repeatedly return to the same comparison block until a condition changes.

10. Understanding Function Call Conventions

Function calls are one of the most important concepts in reverse engineering because programs constantly call functions to perform tasks. Calling conventions define how parameters are passed to functions, where return values are stored, and who is responsible for cleaning the stack after the function finishes executing.

The three common calling conventions encountered in assembly language are:

- `cdecl`
- `stdcall`
- `fastcall`

1. Create the Source Code File

Create the following folder structure on the Desktop:

```
Chapter 6
├── 07_Function_Call_Conventions
│   └── listing_6_16_function_calls
```

Open Visual Studio Code or a text editor of your choice.

Inside the folder:

listing_6_16_function_calls

create a new file named:

listing_6_16_function_calls.c

Paste the following code exactly as shown below:

```
#include <stdio.h>

int test(int x, int y, int z)
{
    return x + y + z;
}

int main()
```

```

{
    int a = 1;
    int b = 2;
    int c = 3;
    int ret;

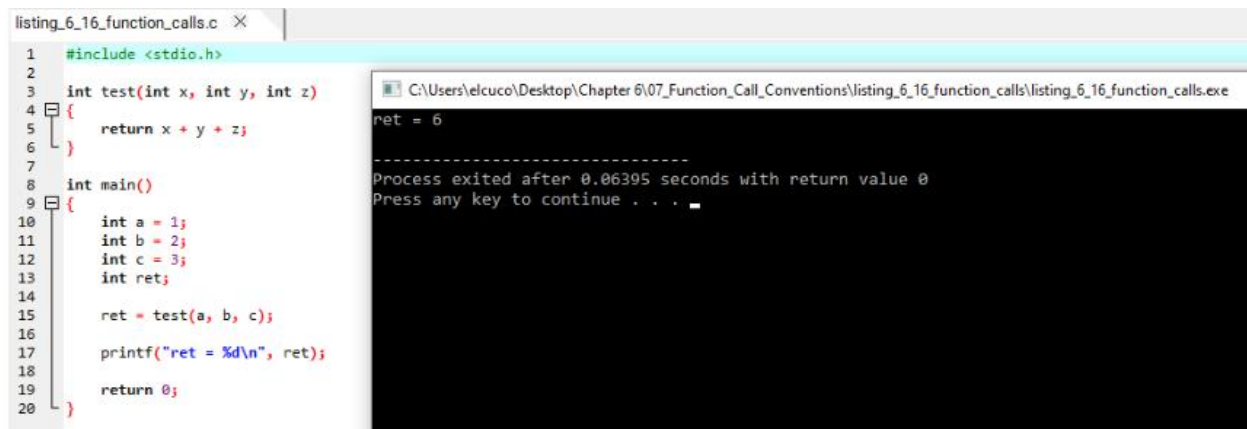
    ret = test(a, b, c);

    printf("ret = %d\n", ret);

    return 0;
}

```

Compile and Run it



The screenshot shows a code editor with a file named `listing_6_16_function_calls.c`. The code defines a function `test` that takes three integers `x`, `y`, and `z` and returns their sum. The `main` function calls `test` with arguments `a=1`, `b=2`, and `c=3`, prints the result, and returns 0. To the right, a terminal window shows the output of the compiled program: `ret = 6`, followed by a message indicating the process exited after 0.06395 seconds with a return value of 0, and a prompt to press any key to continue.

```

listing_6_16_function_calls.c
1  #include <stdio.h>
2
3  int test(int x, int y, int z)
4  {
5      return x + y + z;
6  }
7
8  int main()
9  {
10     int a = 1;
11     int b = 2;
12     int c = 3;
13     int ret;
14
15     ret = test(a, b, c);
16
17     printf("ret = %d\n", ret);
18
19     return 0;
20 }

```

```

C:\Users\elcucu\Desktop\Chapter 6\07_Function_Call_Conventions\listing_6_16_function_calls\listing_6_16_function_calls.exe
ret = 6
-----
Process exited after 0.06395 seconds with return value 0
Press any key to continue . . .

```

2. What this means

The program creates three variables:

```

a = 1
b = 2
c = 3

```

The function call:

`ret = test(a, b, c);` passes the values into the `test()` function.

Inside the function, the values are added together:

1 + 2 + 3 = 6

The result is returned and stored in: `ret`

The program then prints: `ret = 6`

cdecl Calling Convention

The `cdecl` calling convention is one of the most common conventions in C programs.

In `cdecl`:

- parameters are pushed onto the stack from right to left
- the caller cleans the stack
- the return value is stored in `eax`

Example assembly:

```
push c
push b
push a
call test
add esp, 12
mov ret, eax
```

The instruction: `add esp, 12` cleans the stack after the function call.

The value `12` represents:

- 3 parameters
- 4 bytes each
- total = 12 bytes

stdcall Calling Convention

The `stdcall` calling convention is commonly used in the Windows API.

In `stdcall`:

- parameters are pushed onto the stack from right to left
- the called function cleans the stack
- the return value is stored in `eax`

Example assembly:

```
push c
push b
push a
call test
```

Unlike `cdecl`, there is no: `add esp, 12`

after the function call because the callee performs the stack cleanup internally.

fastcall Calling Convention

The `fastcall` calling convention is designed for faster execution.

Instead of pushing all parameters onto the stack, some parameters are passed using CPU registers.

Example assembly:

```
mov ecx, a
mov edx, b
push c
call test
```

In this example:

- `a` is passed through `ecx`
- `b` is passed through `edx`
- `c` is pushed onto the stack

Using registers reduces stack operations and improves performance.

Push vs Move

One of the important observations in calling conventions is the difference between `push` and `mov`.

The instruction: `push eax` places data onto the stack.

This is commonly used in:

- `cdecl`
- `stdcall`

The instruction:

`mov eax, 5` copies data into a register.

This is commonly used in:

- `fastcall`

because parameters are often passed through registers such as:

- `ecx`
- `edx`

instead of the stack.

2. Your key point

The important observation is that calling conventions determine:

- how parameters are passed
- how functions return values
- who cleans the stack
- how registers are used

A reverse engineer can identify calling conventions by analysing:

- `push` instructions
- register usage
- stack cleanup instructions
- function calls
- return values stored in `eax`.

11. Analyzing Switch Statements in Assembly Language

Switch statements are used to make decisions based on a single value. Depending on the value provided, a different block of code executes.

In assembly language, switch statements are commonly compiled in two ways:

- if-style conditional jumps
- jump tables

This example demonstrates the if-style implementation.

1. Create the Source Code File

Create the following folder structure on the Desktop:

```
Chapter 6
├── 08_Switch_Statements
│   └── listing_6_20_switch_statement
```

Inside the folder:

```
listing_6_20_switch_statement
```

create a new file named:

```
listing_6_20_switch_statement.c
```

Paste the following code exactly as shown below:

```
#include <stdio.h>

int test(int x, int y, int z)
{
    return x + y + z;
}

int main()
{
    int a = 1;
    int b = 2;
    int c = 3;
    int ret;

    ret = test(a, b, c);
```

```

printf("ret = %d\n", ret);

return 0;
}

```

Compile the Program

```

listing_6_20_switch_statement.c
1  #include <stdio.h>
2
3  int main()
4  {
5      int i = 2;
6      switch(i)
7      {
8          case 1:
9              printf("i = %d", i + 1);
10             break;
11
12             case 2:
13                 printf("i = %d", i + 2);
14                 break;
15
16             case 3:
17                 printf("i = %d", i + 3);
18                 break;
19
20             default:
21                 break;
22         }
23     }
24     return 0;
25 }

```

```

C:\Users\elcucu\Desktop\Chapter 6\08_Switch_Statements\listing_6_20_switch_statement\listing_6_20_switch_statement.exe
i = 4
-----
Process exited after 0.05646 seconds with return value 0
Press any key to continue . . .

```

2. What this means

The program creates a local variable: `i = 2`

The switch statement checks the value of `i`.

The program then compares `i` against each case:

- case 1
- case 2
- case 3

Since: `i = 2` the program executes:

```
printf("i = %d", i + 2);
```

which prints: `i = 4`

After executing the selected case, the `break` statement exits the switch statement.

3. What to Look for in IDA

In assembly language, switch statements compiled in if-style form usually contain:

- several comparisons
- several conditional jumps
- multiple execution branches

Important instructions usually include:

```
cmp, jz, jnz, jmp
```

4. Explanation

The variable initialization usually appears similar to:

```
mov [rbp+i], 2
```

This stores the value 2 into the local variable `i`.

The switch statement is translated into several comparisons.

Example:

```
cmp [rbp+i], 1
jz short loc_case1
```

This checks whether `i` equals 1.

If not equal, execution continues to the next comparison:

```
cmp [rbp+i], 2
jz short loc_case2
```

Since `i = 2`, this jump is taken.

Execution then moves to the code block for case 2.

Inside that block, the program executes:

```
add ecx, 2
call printf which performs:
i + 2 and prints the result.
```

At the end of each case, an unconditional jump instruction skips the remaining cases and moves to the end of the switch statement.

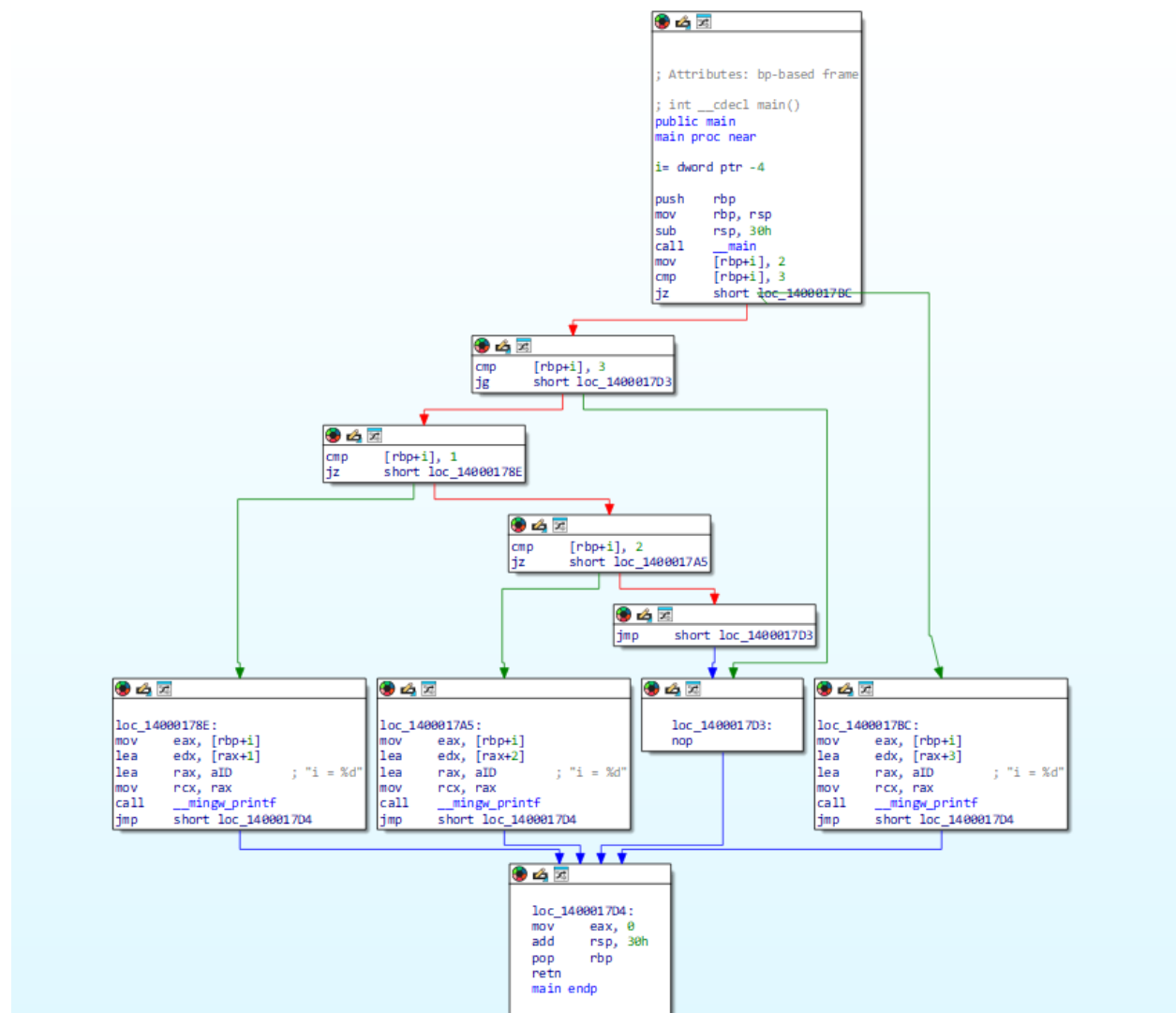
5. Graph View Observation

In IDA graph mode:

- switch statements create multiple branches
- each case appears as a separate execution path
- all paths eventually merge at the end of the function

This structure makes switch statements easy to recognise visually.

Unlike simple if statements, switch statements usually produce many independent branches



6. Your key point

The important observation is that switch statements compiled in if-style form are recognised by:

- repeated comparisons
- repeated conditional jumps
- several independent execution branches

A reverse engineer can identify switch logic by observing multiple comparison-and-jump patterns that lead to separate blocks of code.

Switch Statements in Assembly Language

Switch statements are commonly compiled in two ways:

- If Style
- Jump Table

Both methods allow the program to select code execution paths based on a single variable value.

1. If Style Switch Statements

In the if-style method, the compiler converts the switch statement into multiple comparisons and conditional jumps.

This behaves similarly to several `if` statements connected together.

Example C Code

```
switch(i)
{
    case 1:
        printf("i = %d", i + 1);
        break;
    case 2:
        printf("i = %d", i + 2);
        break;

    case 3:
        printf("i = %d", i + 3);
        break;
}
```

Example Assembly

```
cmp    [rbp+i], 1
jz     short loc_case1

cmp    [rbp+i], 2
jz     short loc_case2

cmp    [rbp+i], 3
jz     short loc_case3
```

Explanation

The program compares the value of `i` against each possible case value.

If: `i == 1` execution jumps to `case 1`

If: `i == 2` execution jumps to `case 2`

If: `i == 3` execution jumps to `case 3`

Each comparison uses:

- `cmp`
- followed by a conditional jump such as:
 - `jz`
 - `je`

This approach creates many comparison instructions and many branches.

Characteristics of If Style

Feature	Description
Method	Multiple comparisons
Instructions	<code>cmp</code> , <code>jz</code> , <code>jne</code>
Best for	Small switch statements
Graph View	Many separate branches

2. Jump Table Switch Statements

For larger switch statements, compilers often use jump tables.

Instead of performing many comparisons, the program uses the switch value as an index into a table of addresses.

The program then jumps directly to the correct case block.

This method is much faster.

Example Concept

Suppose:

`i = 2`

The jump table stores addresses for:

- case 0
- case 1
- case 2
- case 3

The program calculates:

- where case 2 is located
- then jumps directly there

Example Assembly

```
mov    eax, [rbp+i]
cmp    eax, 3
ja     short default_case

jmp     ds:jump_table[eax*4]
```

Explanation

The instruction:

```
mov eax, [rbp+i]
```

loads the switch value into `eax`.

The instruction:

```
cmp eax, 3
```

checks whether the value is within range.

The instruction:

```
jmp ds:jump_table[eax*4]
```

uses the value in `eax` to directly select a memory address from the jump table.

The program then jumps directly to the matching case block.

Characteristics of Jump Tables

Feature	Description
Method	Indexed jump
Instructions	<code>jmp</code> , indexed addressing
Best for	Large switch statements
Speed	Faster
Graph View	Many blocks connected from one jump

Main Difference

If Style	Jump Table
Many comparisons	One indexed jump
Slower	Faster
Easier to read	More complex

Used for small switches Used for large switches

What Reverse Engineers Look For

If Style Indicators

```
cmp
jz
jne
```

repeated many times.

Jump Table Indicators

```
jmp [eax*4]
```

or:

```
jmp ds:table[eax*4]
```

These indicate indexed jumps into a jump table.

7. Your key point

The important observation is that switch statements can appear very differently in assembly language depending on how the compiler optimizes the code.

A reverse engineer must determine whether:

- the switch uses repeated comparisons (if style)
- or direct indexed jumps (jump table)

Both patterns are very common in malware and software analysis.

12. Disassembling Arrays in Assembly Language

Arrays are used to store multiple values of the same data type in sequential memory locations.

In assembly language, arrays are accessed using:

- a base memory address
- an index
- an offset calculation

This makes arrays very important in reverse engineering and malware analysis.

1. Create the Source Code File

Create the following folder structure on the Desktop:

```
Chapter 6
├── 09_Arrays
│   └── listing_6_24_arrays
```

Open Visual Studio Code or a text editor of your choice.

Inside the folder:

```
listing_6_24_arrays
```

create a new file named:

```
listing_6_24_arrays.c
```

Paste the following code exactly as shown below:

```
#include <stdio.h>
```

```
int b[5] = {123,87,487,-7,978};
```

```
int main()
```

```
{
```

```
    int i;
```

```
    int a[5];
```

```
    for(i = 0; i < 5; i++)
```

```
    {
```

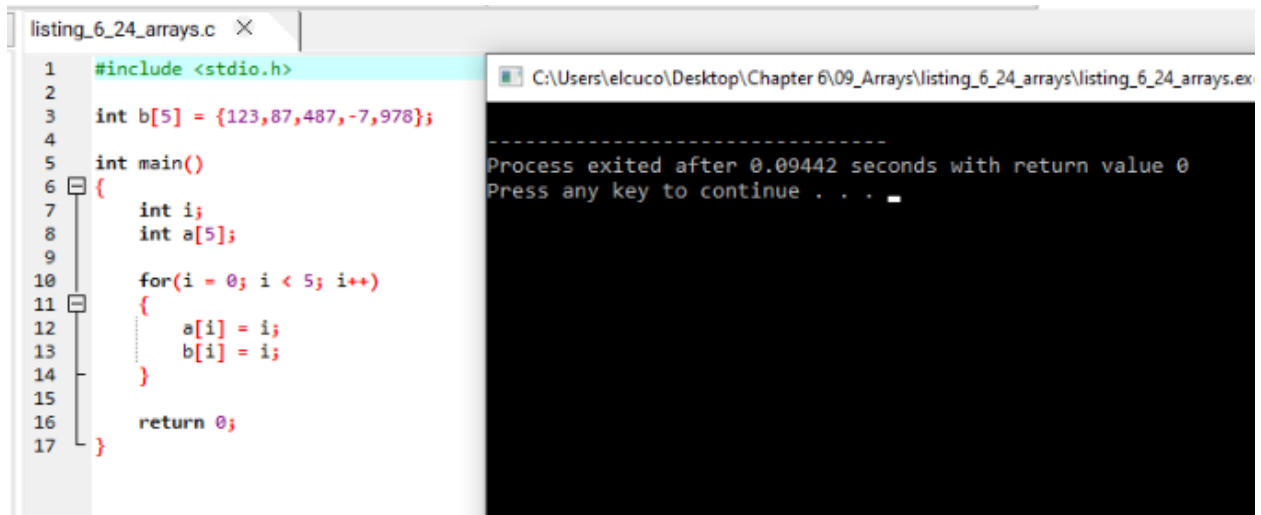
```
        a[i] = i;
```

```
        b[i] = i;
```

```
    }
```

```
    return 0;
}
```

Compile and Run it



```
listing_6_24_arrays.c ×
1  #include <stdio.h>
2
3  int b[5] = {123,87,487,-7,978};
4
5  int main()
6  {
7      int i;
8      int a[5];
9
10     for(i = 0; i < 5; i++)
11     {
12         a[i] = i;
13         b[i] = i;
14     }
15
16     return 0;
17 }
```

```
C:\Users\elcuco\Desktop\Chapter 6\09_Arrays\listing_6_24_arrays\listing_6_24_arrays.exe
-----
Process exited after 0.09442 seconds with return value 0
Press any key to continue . . .
```

The program runs silently because no output is printed.

2. What this means

The program creates:

- a global array named `b`
- a local array named `a`

The global array:

```
int b[5] = {123,87,487,-7,978};
```

is stored in global memory.

The local array:

```
int a[5];
```

 is stored on the stack inside `main()`.

The loop:

```
for(i = 0; i < 5; i++)
```

iterates from:

- 0
- to:
- 4

Inside the loop:

- `a[i] = i`
- `b[i] = i`

This means each array element receives the current value of `i`.

3. What to Look for in IDA

Important instructions usually include:

```
mov
add
cmp
jmp
```

You should also look for:

- indexed memory access
- multiplication by element size
- base addresses

4. Explanation

The instruction:

```
mov [ebp+var_18], 0
```

 initializes the loop counter `i` to `0`.

The loop comparison:

```
cmp [ebp+var_18], 5
```

 checks whether `i` is less than `5`.

Local Array Access

The instruction:

```
mov [ebp+ecx*4+var_14], edx
```

 accesses the local array `a`.

This means:

- `ebp+var_14` is the base address of array `a`
- `ecx` contains the index
- `*4` scales the index because integers use 4 bytes

So: `a[i]` is accessed by: base address + (index × 4)

Global Array Access

The instruction:

```
mov dword_40A000[ecx*4], eax
```

accesses the global array `b`.

Here:

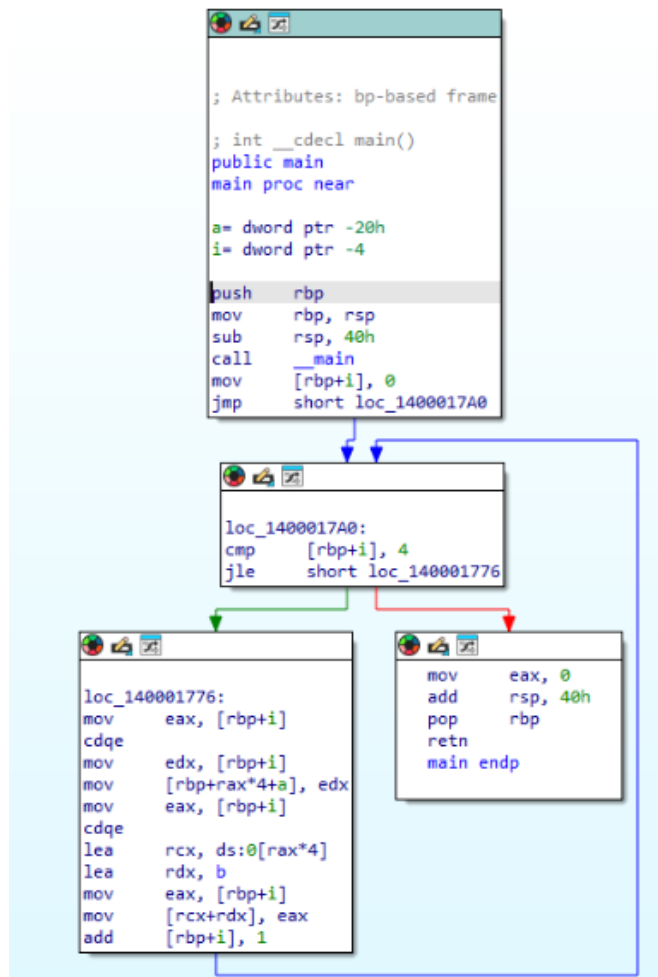
- `dword_40A000` is the base address of array `b`
- `ecx*4` calculates the correct array element offset

This directly accesses global memory.

Difference Between Local and Global Arrays

Array	Location	Assembly Style
a	Stack	<code>[ebp+ecx*4+offset]</code>
b	Global memory	<code>dword_40A000[ecx*4]</code>

5. Graph View Observation



In graph view:

- the loop creates a circular execution path
- array operations occur inside the loop body
- indexed memory access appears repeatedly

This combination of:

- loops
- indexing
- memory offsets

is a strong indicator of array manipulation.

6. Your key point

The important observation is that arrays in assembly are accessed using:

- a base memory address
- an index
- scaled offsets

A reverse engineer can identify arrays by looking for indexed memory access patterns such as:

```
[ebp+ecx*4+offset]    or:    dword_40A000[ecx*4]
```

These patterns indicate that the program is accessing sequential memory locations as array elements.

7. Summary

This chapter demonstrated how common C programming constructs appear in assembly language after compilation.

The practical exercises introduced:

- Variable storage
- Arithmetic operations
- Conditional logic
- Loops
- Function calls
- Switch statements
- Arrays
- Graph analysis in IDA Pro

Understanding these assembly patterns is essential for:

- Reverse engineering
- Malware analysis
- Vulnerability research
- Low-level debugging
- Software analysis

By recognising these patterns, analysts can better understand program behaviour even when source code is **unavailable**.